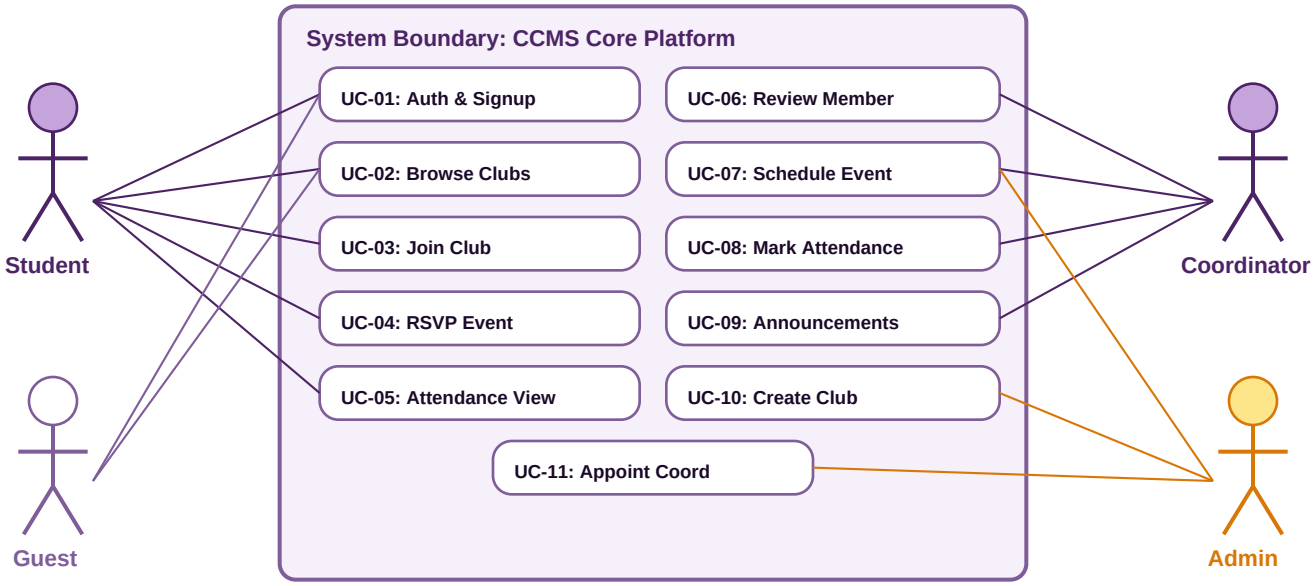
College Club Management System (CCMS)

IA3 Software Engineering Milestone — UML System Design Specification

Version: 1.0.0	**Standard:** UML 2.5 / IEEE 830	**Date:** November 2026	**Target System:** Web / Cloud

1. Use Case Diagram

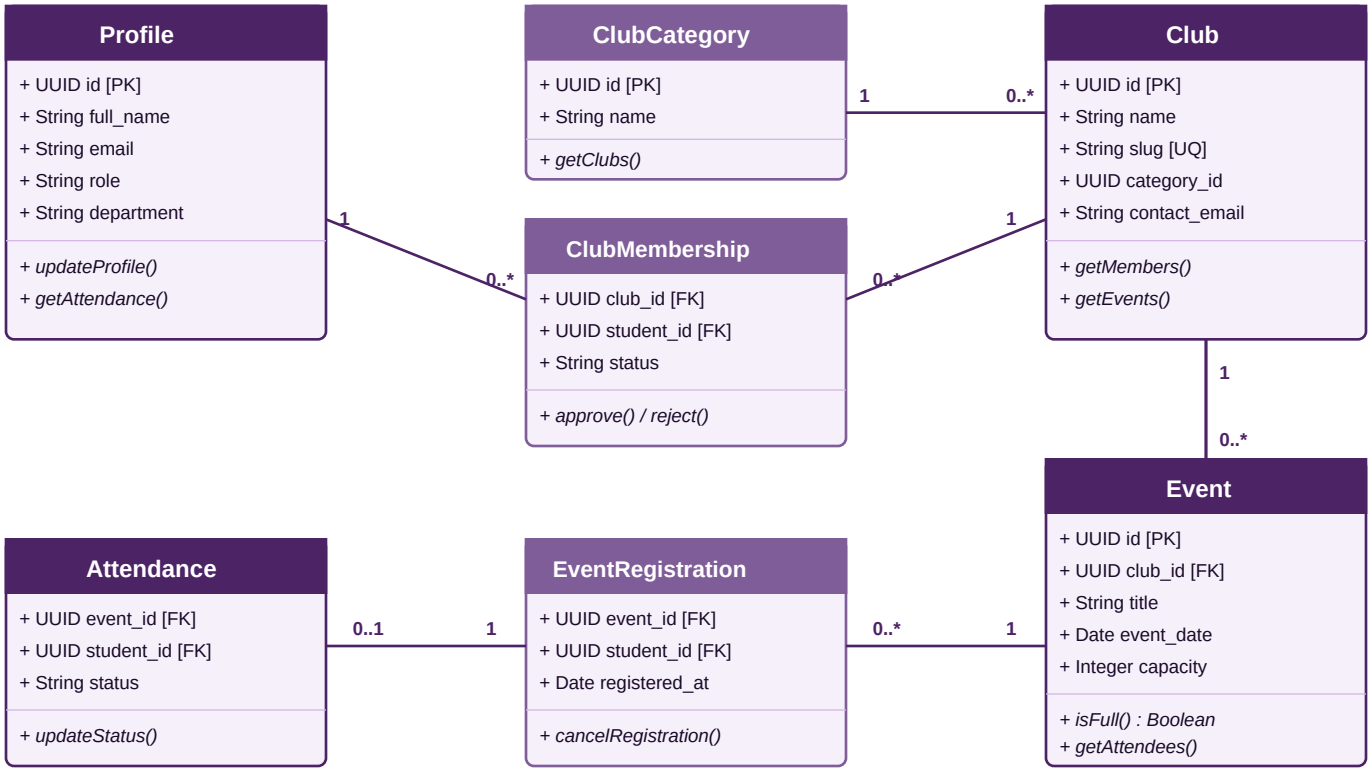
Description: Illustrates system actors (Guest, Student, Coordinator, Admin), the system boundary, and interactions with the core functional use cases.



Actor	Accessible Core Use Cases	Security & System Permissions
Student	UC-01 (Auth), UC-02 (Browse), UC-03 (Join), UC-04 (RSVP), UC-05 (Attendance)	Restricted strictly to personal profile and owned club/event registrations.
Coordinator	UC-06 (Review Members), UC-07 (Events), UC-08 (Attendance), UC-09 (Announcements)	Elevated permissions scoped strictly to assigned club ID via PostgreSQL RLS.
Admin	UC-10 (Create Club), UC-11 (Appoint Coordinator), UC-12 (Analytics & Audit)	Global platform superuser; manages categories, clubs, coordinators, and roles.

2. Class Diagram

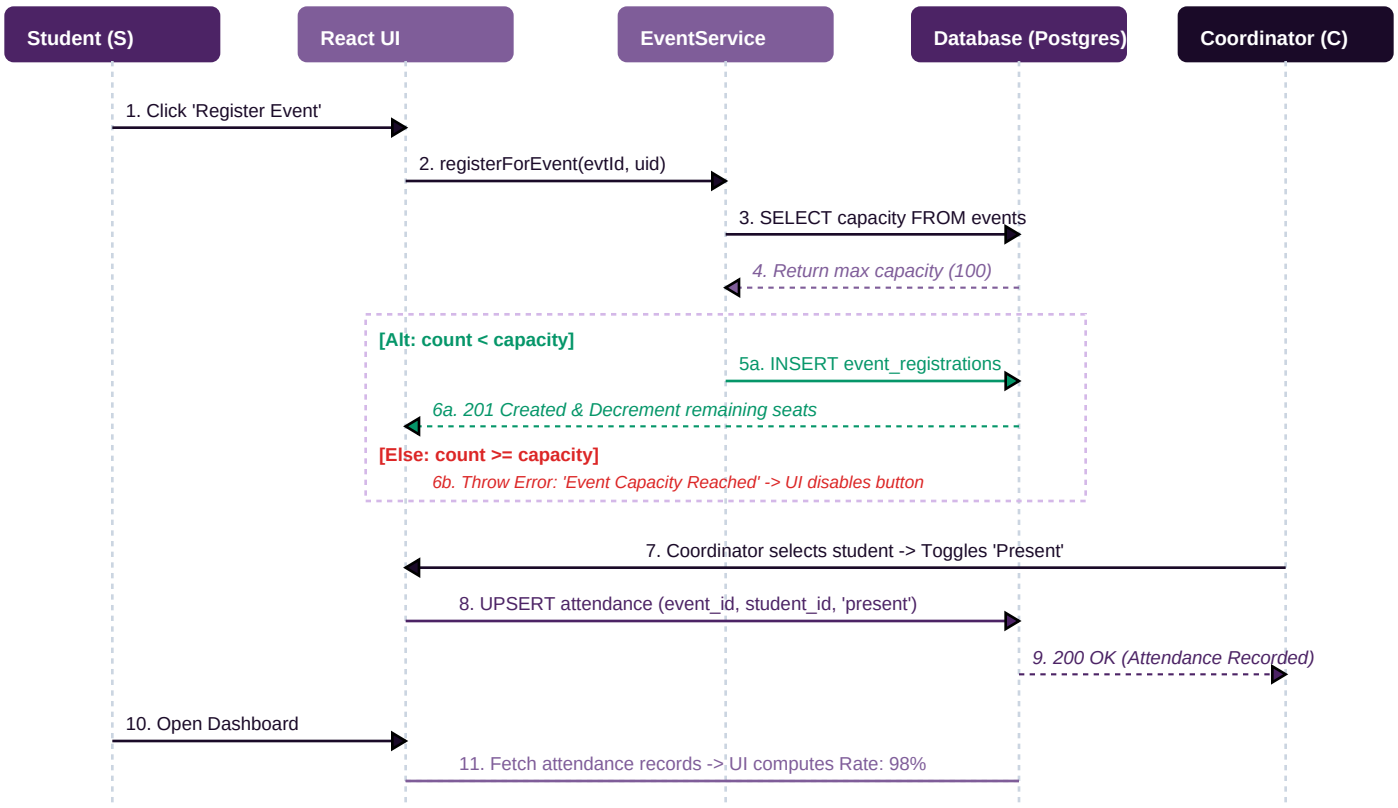
Description: Defines the structural domain model, representing entities, attributes, data types, operations, associations, and multiplicity constraints across the database and application layers.



Multiplicity & Structural Rules: Each Club is associated with exactly 1 ClubCategory. A Profile may establish 0..* ClubMembership rows. An Event receives registrations up to its hard-coded capacity limit, after which subsequent insert attempts are rejected at the service layer.

3. Sequence Diagram

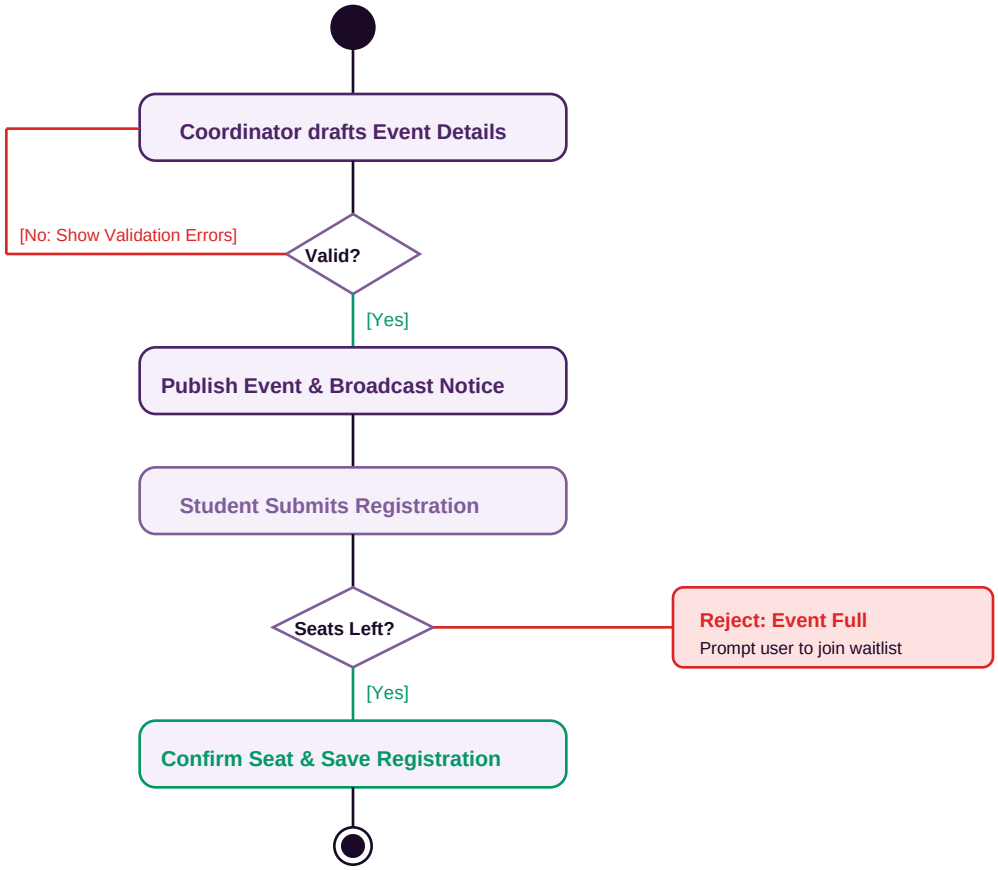
Scenario: Complete Student Event Registration, Capacity Guard Validation, Coordinator Attendance Verification, and Dynamic Attendance Rate Calculation.



Execution Order & Invariants: Step 3 guarantees that race conditions on event caps are guarded prior to record insertion. Steps 8–9 leverage PostgreSQL ON CONFLICT (event_id, student_id) DO UPDATE semantics to prevent duplicate attendance logs.

4. Activity Diagram

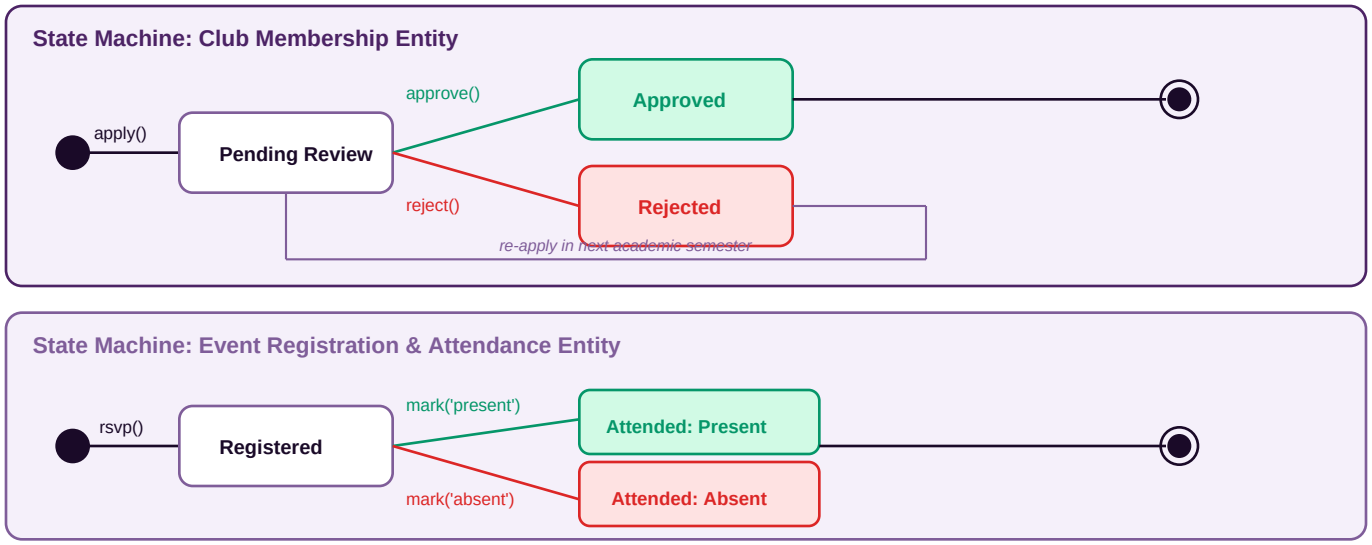
Workflow: High-level operational workflow representing end-to-end event lifecycle management, registration decision diamonds, attendance capture loop, and student transcript synchronization.



Activity Decisions & Exception Handling: All coordinator form inputs are evaluated synchronously before dispatch. The capacity check branches prevent over-subscription by checking real-time database counts before committing attendance eligibility.

5. State Machine Diagram

Entity Lifecycle: Illustrates the operational states, state transitions, triggers, and terminal states for **Club Membership** and **Event Registration** entities.



Entity	Initial State	Trigger Events & Guards	Terminal / Valid States
Club Membership	Unapplied	apply() → Pending; approve() [if Coordinator] → Approved; reject() → Rejected	Approved, Rejected, Cancelled
Event Registration	Unregistered	rsvp() [if count < capacity] → Registered; cancel() → Cancelled	Attended (Present / Late / Absent)